

Università di Pisa

Laurea Magistrale in Informatica

Computational Mathematics for Learning and Data Analysis

Dispensa del corso

Basata sulle lezioni di

Prof. Antonio Frangioni (Ottimizzazione)

Prof. Alice Cortinovis (Analisi Numerica Lineare)

A.A. 2025/2026

Questa dispensa integra le slide ufficiali, le note di Frangioni (2025/26) e gli appunti dalle esercitazioni di Cortinovis. È redatta in italiano con terminologia tecnica in inglese.

Obiettivo: copertura completa di tutto il programma del corso.

Indice

Introduzione al Corso	7
I Ottimizzazione Matematica	9
1 Semplici Problemi di Ottimizzazione	11
1.1 Formulazione generale	11
1.1.1 Minimi locali vs globali	11
1.2 Ottimizzazione univariata (1D)	11
1.2.1 Golden Section Search	12
1.2.2 Metodo di Newton 1D	12
2 Ottimizzazione Non Vincolata	13
2.1 Richiami di analisi in più variabili	13
2.1.1 Gradiente, Hessiana, sviluppo di Taylor	13
2.1.2 Condizioni di ottimalità	13
2.1.3 Classi di regolarità importanti	14
2.1.4 Funzioni convesse	14
2.2 Metodi del gradiente (smooth)	14
2.2.1 Gradient Descent a passo fisso	14
2.2.2 Line Search adattiva	16
2.3 Metodo di Newton multivariato	16
2.4 Metodi quasi-Newton	17
2.4.1 Aggiornamento BFGS	17
2.5 Gradiente Coniugato per funzioni quadratiche	17
3 Ottimizzazione Non Vincolata — Metodi non-smooth	19
3.1 Funzioni non differenziabili	19
3.2 Metodo del subgradiente	19
3.3 Metodo del gradiente prossimale	20
3.4 Metodi bundle	20
4 Ottimizzazione Vincolata	23
4.1 Formulazione e topologia	23
4.2 Condizioni KKT	23

4.2.1	Vincoli attivi e LICQ	23
4.2.2	Lemma di Farkas e dualità	24
4.3	Dualità Lagrangiana	24
4.4	Algoritmi per l'ottimizzazione vincolata	25
4.4.1	Gradiente proiettato	25
4.4.2	Metodo di Frank-Wolfe (Conditional Gradient)	25
4.4.3	Metodi barriera (Interior Point)	25
II	Analisi Numerica Lineare (NLA)	27
5	Background: Algebra Lineare	29
5.1	Prodotto matriciale e interpretazioni	29
5.2	Sistemi triangolari	29
5.3	Autovalori e forme quadratiche	29
6	Norme e Matrici Ortogonali	31
6.1	Norme vettoriali	31
6.2	Matrici ortogonali	31
6.3	Norme di matrici	31
7	Decomposizione a Valori Singolari (SVD)	33
7.1	Definizione e proprietà	33
7.2	Teorema di Eckart-Young (approssimazione di basso rango)	34
8	Il Problema dei Minimi Quadrati	35
8.1	Formulazione	35
8.2	Equazioni normali	35
8.3	Soluzione via SVD	35
9	Fattorizzazione QR	37
9.1	Definizione e teorema	37
9.2	Riflettori di Householder	37
9.3	Soluzione dei minimi quadrati con QR	38
10	Condizionamento e Stabilità	39
10.1	Numero di condizionamento	39
10.2	Aritmetica floating point	39
10.3	Stabilità degli algoritmi	40
10.3.1	Test a posteriori: analisi del residuo	41
11	Metodi Iterativi per Sistemi Lineari	43
11.1	Motivazione: sistemi sparsi di grande scala	43
11.2	Spazi di Krylov	43
11.3	Gradiente Coniugato per sistemi SPD	43

11.3.1 Precondizionamento	43
11.4 Algoritmo di Arnoldi	44
11.5 GMRES	44
12 Fattorizzazioni Dirette per Sistemi Lineari	47
12.1 Fattorizzazione LU	47
12.2 Fattorizzazione $LDL^\top$ per matrici simmetriche	48
12.3 Fattorizzazione di Cholesky	48
12.4 Confronto e scelta dell'algoritmo	48
12.5 Esempi pratici	48
13 Riepilogo: Connessioni tra NLA e Ottimizzazione	51
A Appendice: Richiami e Tabelle di Riferimento	53
A.1 Notazione	53
A.2 Costi computazionali principali	54
A.3 Tassi di convergenza degli algoritmi di ottimizzazione	54
A.4 Regole pratiche per l'esame	54

Introduzione al Corso

Il corso di **Computational Mathematics for Learning and Data Analysis** è tenuto da due professori:

- **Prof. Antonio Frangioni** — parte di *ottimizzazione*: risoluzione di $\min_{x \in S} f(x)$ per classi crescenti di funzioni e insiemi ammissibili.
- **Prof. Alice Cortinovis** — parte di *analisi numerica lineare (NLA)*: risoluzione efficiente e accurata del problema dei minimi quadrati $\min_x \|Ax - b\|^2$ e dei sistemi lineari.

Le due parti sono strettamente interconnesse: molti algoritmi di ottimizzazione richiedono la risoluzione di sistemi lineari come sotto-passo (es. il passo di Newton). Capire entrambe è fondamentale.

Prerequisiti: algebra lineare (spazi vettoriali, autovalori), analisi (Taylor, derivate parziali), nozioni di programmazione (Matlab/Python).

Parte I

Ottimizzazione Matematica

Capitolo 1

Semplici Problemi di Ottimizzazione

1.1 Formulazione generale

Definizione 1.1 (Problema di ottimizzazione). Dato un insieme *ammissibile* $S \subseteq \mathbb{R}^n$ e una funzione *obiettivo* $f : S \rightarrow \mathbb{R}$, il problema è:

$$f^* = \min_{x \in S} f(x).$$

Un punto $x^* \in S$ tale che $f(x^*) = f^*$ è una *soluzione ottima* (o *minimizzatore*).

Osservazione 1.1. Il valore ottimo f^* è unico se esiste, ma possono esistere più minimizzatori x^* . Massimizzare $g(x)$ equivale a minimizzare $f(x) = -g(x)$. Un punto x_ε è una *soluzione ε -ottima* se $f(x_\varepsilon) \leq f^* + \varepsilon$.

1.1.1 Minimi locali vs globali

Definizione 1.2. x^* è un *minimo locale* se esiste $\delta > 0$ tale che $f(x^*) \leq f(x)$ per ogni $x \in S$ con $\|x - x^*\| \leq \delta$. È un *minimo globale* se $f(x^*) \leq f(x)$ per ogni $x \in S$.

Per funzioni convesse, ogni minimo locale è globale. Per funzioni non convesse, possono esistere minimi locali che non sono globali — trovare il globale è in generale NP-difficile.

1.2 Ottimizzazione univariata (1D)

Il caso $n = 1$ è fondamentale perché molte line search si riducono a questo.

Definizione 1.3 (Funzione unimodale). $f : [a, b] \rightarrow \mathbb{R}$ è unimodale se esiste $x^* \in [a, b]$ tale che f è strettamente decrescente su $[a, x^*]$ e strettamente crescente su $[x^*, b]$.

1.2.1 Golden Section Search

Algoritmo: Golden Section Search

Dato f unimodale su $[a, b]$, con $\tau = (\sqrt{5} - 1)/2 \approx 0.618$:

1. $x_1 = a + (1 - \tau)(b - a)$, $x_2 = a + \tau(b - a)$.
2. Se $f(x_1) < f(x_2)$: poni $b \leftarrow x_2$, altrimenti $a \leftarrow x_1$.
3. Aggiorna il punto interno rimanente senza ricalcolarlo (1 valutazione per passo).
4. Ripeti finché $|b - a| \leq \varepsilon$.

Dopo k passi, l'intervallo si è ridotto a $(b - a) \cdot \tau^k$. Costo: $O(\log(1/\varepsilon))$ valutazioni.

Intuizione 1.1. La scelta di $\tau = (\sqrt{5}-1)/2$ (rapporto aureo) garantisce che il punto “risparmiato” cada nella posizione giusta nel sotto-intervallo successivo: un’elegante proprietà di auto-similarità.

1.2.2 Metodo di Newton 1D

Per $f \in C^2$, partendo da x_0 :

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Convergenza *quadratica* ($\|e_{k+1}\| = O(\|e_k\|^2)$) vicino a una radice di f' con $f''(x^*) \neq 0$.

Capitolo 2

Ottimizzazione Non Vincolata

2.1 Richiami di analisi in più variabili

2.1.1 Gradiente, Hessiana, sviluppo di Taylor

Definizione 2.1 (Gradiente). Per $f : \mathbb{R}^n \rightarrow \mathbb{R}$ differenziabile:

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)^\top \in \mathbb{R}^n.$$

$\nabla f(x)$ punta nella *direzione di massima crescita* di f in x .

Definizione 2.2 (Hessiana). Per $f \in C^2(\mathbb{R}^n)$:

$$\nabla^2 f(x) = H_f(x) \in \mathbb{R}^{n \times n}, \quad [H_f(x)]_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}(x).$$

Per il teorema di Schwarz, H_f è simmetrica.

Sviluppo di Taylor al secondo ordine:

$$f(x+d) = f(x) + \nabla f(x)^\top d + \frac{1}{2} d^\top H_f(x) d + o(\|d\|^2).$$

Esempio 2.1 (Funzione quadratica). $f(x) = \frac{1}{2} x^\top Q x - c^\top x$ con Q simmetrica: $\nabla f(x) = Qx - c$, $H_f(x) = Q$ (costante).

2.1.2 Condizioni di ottimalità

Teorema 2.1 (Condizioni del primo e secondo ordine). Per $f \in C^2$:

1. **Necessaria 1° ordine:** Se x^* è minimo locale, $\nabla f(x^*) = 0$ (punto stazionario).
2. **Necessaria 2° ordine:** Se x^* è minimo locale, $H_f(x^*) \succeq 0$.
3. **Sufficiente:** Se $\nabla f(x^*) = 0$ e $H_f(x^*) \succ 0$, allora x^* è minimo locale isolato.

2.1.3 Classi di regolarità importanti

Definizione 2.3 (L -smooth). $f \in C^1$ è L -smooth se ∇f è Lipschitz con costante $L > 0$:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y.$$

Equivalentemente (per $f \in C^2$): $\nabla^2 f(x) \preceq LI$ per ogni x .

Proposizione 2.1 (Lemma di discesa). Se f è L -smooth, allora per ogni x, d :

$$f(x + d) \leq f(x) + \nabla f(x)^\top d + \frac{L}{2}\|d\|^2.$$

Questo è il modello quadratico superiore di f .

Definizione 2.4 (τ -strongly convex). f è fortemente convessa con modulo $\tau > 0$ se:

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\tau}{2}\|y - x\|^2 \quad \forall x, y.$$

Equivalentemente: $\nabla^2 f(x) \succeq \tau I$, cioè tutti gli autovalori dell'Hessiana sono $\geq \tau$.

Osservazione 2.1. La classe “più conveniente” per l'ottimizzazione è f L -smooth e τ -fortemente convessa, con $\kappa = L/\tau$ (numero di condizionamento dell'ottimizzazione). Se $\kappa \gg 1$, il problema è *difficile*.

2.1.4 Funzioni convesse

Definizione 2.5 (Funzione convessa). f è convessa se per ogni x, y e $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

Proposizione 2.2 (Caratterizzazioni equivalenti per $f \in C^2$). Le seguenti sono equivalenti:

(a) f convessa; (b) $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$ (disuguaglianza del gradiente); (c) $H_f(x) \succeq 0$ per ogni x .

Teorema 2.2 (Ottimalità globale). Se f è convessa, ogni minimo locale è globale. Se f è strettamente convessa, il minimizzatore è unico.

2.2 Metodi del gradiente (smooth)

2.2.1 Gradient Descent a passo fisso

Algoritmo: Gradient Descent

Dato $x_0 \in \mathbb{R}^n$, step size $\alpha > 0$:

$$x_{k+1} = x_k - \alpha \nabla f(x_k), \quad k = 0, 1, 2, \dots$$

Intuizione 2.1. $-\nabla f(x_k)$ è la direzione di massima discesa. Muoversi di un passo α in tale direzione riduce f di $\alpha\|\nabla f(x_k)\|^2$ (approssimativamente), se α è abbastanza piccolo.

2.2.1.1 Convergenza per funzioni quadratiche

Teorema 2.3 (Convergenza GD per f quadratica $Q \succ 0$). Sia $f(x) = \frac{1}{2}x^\top Qx - c^\top x$ con $0 < \lambda_{\min} = \lambda_n \leq \lambda_1 = \lambda_{\max}$ autovalori di Q . Con passo ottimo $\alpha^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}$:

$$\|x_k - x^*\|_Q \leq \left(\frac{\kappa(Q) - 1}{\kappa(Q) + 1} \right)^k \|x_0 - x^*\|_Q,$$

dove $\kappa(Q) = \lambda_{\max}/\lambda_{\min}$ e $\|v\|_Q^2 = v^\top Qv$.

Osservazione 2.2 (Disuguaglianza di Kantorovich). Nell'analisi di convergenza, la chiave è la *disuguaglianza di Kantorovich*:

$$\frac{(g^\top g)^2}{(g^\top Qg)(g^\top Q^{-1}g)} \geq \frac{4\lambda_{\min}\lambda_{\max}}{(\lambda_{\min} + \lambda_{\max})^2} = 1 - \left(\frac{\kappa - 1}{\kappa + 1} \right)^2.$$

Essa misura quanto la curvatura non uniforme di f rallenti la discesa.

2.2.1.2 Convergenza per funzioni smooth convesse

Teorema 2.4 (GD con passo $1/L$: convergenza $O(1/k)$). Sia f convessa e L -smooth. Con $\alpha = 1/L$:

$$f(x_k) - f^* \leq \frac{L\|x_0 - x^*\|^2}{2k}.$$

Prova (sketch): Dal lemma di discesa con $\alpha = 1/L$:

$$f(x_{k+1}) \leq f(x_k) - \frac{1}{2L}\|\nabla f(x_k)\|^2.$$

Sommando k passi e usando la disuguaglianza del gradiente, si ottiene la tesi.

Teorema 2.5 (GD per funzioni fortemente convesse: convergenza lineare). Se f è L -smooth e τ -fortemente convessa, con $\alpha = 1/L$:

$$\|x_k - x^*\|^2 \leq \left(1 - \frac{\tau}{L}\right)^k \|x_0 - x^*\|^2 = \left(1 - \frac{1}{\kappa}\right)^k \|x_0 - x^*\|^2.$$

La convergenza è lineare (geometrica) con fattore $\rho = 1 - 1/\kappa$.

Osservazione 2.3 (Confronto tra i tassi). $O(1/k)$ (convex) vs $O(\rho^k)$ (strongly convex): il secondo è *esponenzialmente* più veloce. Tuttavia, per κ grande, anche la convergenza lineare è molto lenta ($\rho \approx 1 - 1/\kappa$ piccolo).

2.2.1.3 Metodi con momento: Nesterov

Il metodo di Nesterov (1983) aggiunge un termine di “inerzia”:

$$y_{k+1} = x_k - \frac{1}{L}\nabla f(x_k), \quad x_{k+1} = y_{k+1} + \frac{k-1}{k+2}(y_{k+1} - y_k).$$

Converge come $O(L/k^2)$ invece di $O(L/k)$: *optimal* per la classe dei metodi del primo ordine. Il termine di momento riutilizza la direzione dell'iterazione precedente.

2.2.2 Line Search adattiva

2.2.2.1 Condizioni di Armijo e Wolfe

Definizione 2.6 (Condizione di Armijo (sufficient decrease)). Con direzione di discesa d_k e step $\alpha > 0$:

$$f(x_k + \alpha d_k) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^\top d_k, \quad c_1 \in (0, 1).$$

Definizione 2.7 (Condizione di curvatura (Wolfe)).

$$\nabla f(x_k + \alpha d_k)^\top d_k \geq c_2 \nabla f(x_k)^\top d_k, \quad c_2 \in (c_1, 1).$$

Algoritmo: Backtracking Armijo

1. $\alpha \leftarrow \alpha_0$ (es. $\alpha_0 = 1$), $\rho \in (0, 1)$ (es. $\rho = 0.5$).
2. Finché Armijo non è soddisfatta: $\alpha \leftarrow \rho \alpha$.
3. Usa α per il passo $x_{k+1} = x_k + \alpha d_k$.

Teorema 2.6 (Convergenza GD con Armijo). Se f è L -smooth e inferiormente limitata, e si usa *backtracking Armijo*, allora $\|\nabla f(x_k)\| \rightarrow 0$.

Osservazione 2.4 (Line search esatta per quadratiche). Per $f(x) = \frac{1}{2}x^\top Qx - c^\top x$, $g_k = Qx_k - c$:

$$\alpha_k^* = \arg \min_{\alpha \geq 0} f(x_k - \alpha g_k) = \frac{g_k^\top g_k}{g_k^\top Q g_k}.$$

2.3 Metodo di Newton multivariato

Algoritmo: Metodo di Newton

Dato x_0 , per $k = 0, 1, \dots$:

$$H_f(x_k)d_k = -\nabla f(x_k), \quad x_{k+1} = x_k + d_k.$$

(con eventuale line search: $x_{k+1} = x_k + \alpha_k d_k$)

Intuizione 2.2. Newton *minimizza il modello quadratico locale* di f :

$$m_k(d) = f(x_k) + \nabla f(x_k)^\top d + \frac{1}{2}d^\top H_f(x_k)d.$$

Il minimo di m_k (se $H_f \succ 0$) è esattamente $d_k = -H_f^{-1}\nabla f$.

Teorema 2.7 (Convergenza quadratica locale). Se $f \in C^3$, x^* è un minimo con $H_f(x^*) \succ 0$, e x_0 è sufficientemente vicino a x^* :

$$\|x_{k+1} - x^*\| \leq C \|x_k - x^*\|^2$$

per una costante $C > 0$ che dipende da $\|\nabla^3 f\|$ e da $\lambda_{\min}(H_f(x^*))$.

Limitazioni: Costo $O(n^3)$ per passo (fattorizzazione di H_f); H_f potrebbe non essere $\succ 0$ lontano da x^* ; non globalmente convergente senza modifiche.

Modifiche pratiche:

- **Damped Newton:** $x_{k+1} = x_k + \alpha_k d_k$ con line search su α_k .
- **Regularized Newton:** $H_f(x_k) + \mu_k I$ con $\mu_k > 0$ (connessione con trust-region).
- **Inexact Newton:** risolvere il sistema $H_f d = -\nabla f$ approssimativamente (es. con CG).

2.4 Metodi quasi-Newton

Idea: approssimare H_f^{-1} tramite aggiornamenti di rango basso, usando solo informazioni sul gradiente.

Definizione 2.8 (Condizione secante). Dati $s_k = x_{k+1} - x_k$ e $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, si vuole $B_{k+1}s_k = y_k$ (approssimazione $B_{k+1} \approx H_f$).

2.4.1 Aggiornamento BFGS

L'aggiornamento **BFGS** (Broyden–Fletcher–Goldfarb–Shanno), con $H_k \approx H_f^{-1}(x_k)$:

$$H_{k+1} = \left(I - \rho_k s_k y_k^\top\right) H_k \left(I - \rho_k y_k s_k^\top\right) + \rho_k s_k s_k^\top, \quad \rho_k = \frac{1}{y_k^\top s_k}. \quad (2.1)$$

Proposizione 2.3 (Proprietà BFGS). 1. Se $H_k \succ 0$ e $y_k^\top s_k > 0$ (garantito da Wolfe), allora $H_{k+1} \succ 0$.

2. $H_{k+1}s_k = s_k / (y_k^\top s_k) \cdot (y_k^\top s_k) = s_k \dots$ ovvero $H_{k+1}y_k = s_k$ (condizione secante).

3. BFGS è l'aggiornamento di minima perturbazione (in norma di Frobenius pesata) che soddisfa la condizione secante.

Teorema 2.8 (Convergenza superlineare di BFGS). Sotto ipotesi di regolarità standard (f L -smooth, τ -strongly convex), BFGS ha convergenza superlineare:

$$\|x_{k+1} - x^*\| = o(\|x_k - x^*\|).$$

Osservazione 2.5 (L-BFGS per grande scala). L-BFGS (Limited-memory BFGS) memorizza solo le ultime m coppie (s_k, y_k) per ricostruire H_k^{-1} implicitamente. Costo $O(mn)$ per passo (anziché $O(n^2)$ per BFGS pieno). Fondamentale per ML: addestramento di reti neurali, ottimizzazione su $n = 10^7$ parametri.

2.5 Gradiente Coniugato per funzioni quadratiche

Il metodo del *gradiente coniugato* (CG) minimizza $f(x) = \frac{1}{2}x^\top Qx - c^\top x$ con $Q \succ 0$, equivalente a risolvere $Qx = c$.

Definizione 2.9 (Q -coniugazione). Vettori $d_0, d_1, \dots$ sono Q -coniugati se $d_i^\top Q d_j = 0$ per $i \neq j$.

Algoritmo: Gradiente Coniugato (CG)

x_0 dato, $r_0 = c - Qx_0$, $d_0 = r_0$. Per $k = 0, 1, \dots$:

$$\begin{aligned} \alpha_k &= \frac{r_k^\top r_k}{d_k^\top Q d_k}, & x_{k+1} &= x_k + \alpha_k d_k, \\ r_{k+1} &= r_k - \alpha_k Q d_k, & \beta_{k+1} &= \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}, \\ d_{k+1} &= r_{k+1} + \beta_{k+1} d_k. \end{aligned}$$

Teorema 2.9 (Ottimalità di CG). Dopo k passi, x_k minimizza f sullo spazio di Krylov

$$\mathcal{K}_k(Q, r_0) = \text{span}(r_0, Qr_0, \dots, Q^{k-1}r_0).$$

CG converge in al più n passi (terminazione esatta).

Teorema 2.10 (Convergenza lineare di CG).

$$\frac{\|x_k - x^*\|_Q}{\|x_0 - x^*\|_Q} \leq 2 \left(\frac{\sqrt{\kappa(Q)} - 1}{\sqrt{\kappa(Q)} + 1} \right)^k.$$

CG è molto più veloce del gradient descent: il fattore è $\sqrt{\kappa}$ anziché κ .

Intuizione 2.3. CG minimizza il residuo relativo su uno spazio polinomiale: trova il polinomio p_k di grado $\leq k$ con $p_k(0) = 1$ che minimizza $\max_{\lambda_i} |p_k(\lambda_i)|$. Se gli autovalori di Q sono raggruppati in m cluster, CG converge in m passi.

Capitolo 3

Ottimizzazione Non Vincolata — Metodi non-smooth

3.1 Funzioni non differenziabili

Molte funzioni nel ML non sono differenziabili ovunque:

- $f(x) = \|x\|_1 = \sum_i |x_i|$ (LASSO, sparsità, compressive sensing).
- $f(x) = \max_j g_j(x)$ (SVM, ottimizzazione worst-case).
- $f(x) = \max(0, x)$ (ReLU, attivazione neurale).

Definizione 3.1 (Subgradiente). $g \in \mathbb{R}^n$ è un *subgradiente* di f convessa in x se:

$$f(y) \geq f(x) + g^\top(y - x) \quad \forall y.$$

L'insieme dei subgradienti $\partial f(x)$ è detto *subdifferenziale*.

Esempio 3.1. Per $f(x) = |x|$: $\partial f(x) = \{+1\}$ se $x > 0$, $[-1, +1]$ se $x = 0$, $\{-1\}$ se $x < 0$. Per $f(x) = \|x\|_1$: $[\partial f(x)]_i = \text{sign}(x_i)$ se $x_i \neq 0$, e $\in [-1, +1]$ se $x_i = 0$.

Teorema 3.1. x^* è un minimo globale di f convessa $\Leftrightarrow 0 \in \partial f(x^*)$.

3.2 Metodo del subgradiente

Algoritmo: Subgradient Method

Dato x_0 , step sizes $\{\alpha_k\}$:

$$x_{k+1} = x_k - \alpha_k g_k, \quad g_k \in \partial f(x_k).$$

Attenzione

A differenza del gradient descent, il subgradiente **non garantisce** $f(x_{k+1}) < f(x_k)$: l'obiettivo può aumentare da un'iterazione all'altra. Si usa il record del best so far:
 $f_k^{\text{best}} = \min_{i \leq k} f(x_i)$.

Teorema 3.2 (Convergenza del subgradiente). *Se f convessa, Lipschitz con costante G :*

- Con $\alpha_k = c/\sqrt{k}$ (*diminishing*): $f_k^{\text{best}} - f^* \leq \frac{G\|x_0 - x^*\|}{\sqrt{k}}$ (*tasso $O(1/\sqrt{k})$*).
- Con $\alpha_k = c$ costante: $\liminf f_k^{\text{best}} \leq f^* + Gc/2$.

Osservazione 3.1. Il tasso $O(1/\sqrt{k})$ è *ottimale* per metodi del primo ordine su funzioni Lipschitz non smooth. Per funzioni smooth, si può fare meglio: $O(1/k)$ con GD.

3.3 Metodo del gradiente prossimale

Molte funzioni hanno la forma $f(x) = h(x) + r(x)$, con h smooth e r non-smooth ma semplice (es. $r(x) = \lambda\|x\|_1$).

Definizione 3.2 (Operatore prossimale). Per $r : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ e $\alpha > 0$:

$$\text{prox}_{\alpha r}(v) = \arg \min_x \left(r(x) + \frac{1}{2\alpha} \|x - v\|^2 \right).$$

Esempio 3.2 (Prox dell' ℓ^1 : soft thresholding). Per $r(x) = \lambda\|x\|_1$:

$$[\text{prox}_{\alpha\lambda\|\cdot\|_1}(v)]_i = \text{sign}(v_i) \max(|v_i| - \alpha\lambda, 0).$$

Questa è la *sogliatura morbida* (soft thresholding).

Algoritmo: Proximal Gradient (ISTA)

Per $f = h + r$ con h L -smooth, r convessa:

$$x_{k+1} = \text{prox}_{\alpha r}(x_k - \alpha \nabla h(x_k)), \quad \alpha = 1/L.$$

Teorema 3.3 (Convergenza ISTA). Con $\alpha = 1/L$: $f(x_k) - f^* \leq \frac{L\|x_0 - x^*\|^2}{2k}$ (*tasso $O(1/k)$*).

Osservazione 3.2 (FISTA — accelerazione di Nesterov). FISTA (Fast ISTA) aggiunge un passo di momento come Nesterov e raggiunge $O(1/k^2)$, ottimale per la classe. È il metodo standard per LASSO di grande scala.

3.4 Metodi bundle

I metodi bundle sono la generalizzazione dei metodi del gradiente coniugato per funzioni non-smooth.

Definizione 3.3 (Modello bundle (piecewise-linear inferior)). Dati gli iterati precedenti $x^1, \dots, x^i$ con subgradienti $g^j \in \partial f(x^j)$, il *bundle* è:

$$f_B(x) = \max_{j \leq i} \left\{ f(x^j) + (g^j)^\top (x - x^j) \right\} \leq f(x).$$

f_B è un modello *inferiore* di f .

Algoritmo: Bundle Method (schema generale)

Dati x_0 , parametro di prossimità $\mu > 0$:

1. Risolvi: $d^* = \arg \min_d \{f_B(x_k + d) + \frac{\mu}{2}\|d\|^2\}$.
2. Se $\|d^*\| \leq \varepsilon$: STOP (criterio di ottimalità).
3. Valuta $f(x_k + d^*)$ e $g^* \in \partial f(x_k + d^*)$.
4. **Serious step**: se $f(x_k + d^*) \leq f(x_k) - m_1(f_B(x_k + d^*) - f(x_k))$: $x_{k+1} \leftarrow x_k + d^*$.
5. **Null step**: altrimenti $x_{k+1} \leftarrow x_k$. Aggiorna il bundle con il nuovo punto.

Teorema 3.4 (Convergenza dei metodi bundle). *Per f convessa Lipschitz, con μ fisso e bounded: se $f^* > -\infty$, allora $\|d_i^*\| \rightarrow 0$, il che implica la convergenza all'ottimo.*

Capitolo 4

Ottimizzazione Vincolata

4.1 Formulazione e topologia

Definizione 4.1 (Problema vincolato).

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{s.t.} \quad g_i(x) \leq 0, \quad i = 1, \dots, m; \quad h_j(x) = 0, \quad j = 1, \dots, p.$$

Teorema 4.1 (Weierstrass). *Se f è continua e $S \subseteq \mathbb{R}^n$ è compatto e non vuoto, il minimo è attinto.*

Osservazione 4.1. Per il problema vincolato, un punto stazionario di f su $\mathbb{R}^n$ potrebbe essere inammissibile. Le condizioni di ottimalità richiedono una nozione diversa.

4.2 Condizioni KKT

4.2.1 Vincoli attivi e LICQ

Definizione 4.2 (Vincoli attivi). All'ottimo x^* , il vincolo g_i è *attivo* se $g_i(x^*) = 0$, altrimenti *inattivo* ($g_i(x^*) < 0$).

Definizione 4.3 (LICQ — Linear Independence Constraint Qualification). Condizione LICQ: i gradienti dei vincoli attivi in x^* sono linearmente indipendenti.

Teorema 4.2 (Condizioni KKT — Karush-Kuhn-Tucker). *Se x^* è un minimo locale e vale LICQ, esistono moltiplicatori $\mu^* \in \mathbb{R}^m$, $\lambda^* \in \mathbb{R}^p$ tali che:*

$$\begin{aligned} \nabla f(x^*) + \sum_{i=1}^m \mu_i^* \nabla g_i(x^*) + \sum_{j=1}^p \lambda_j^* \nabla h_j(x^*) &= 0 && \text{(stazionarietà)} \\ g_i(x^*) &\leq 0, \quad h_j(x^*) = 0 && \text{(ammissibilità)} \\ \mu_i^* &\geq 0 && \text{(non negatività)} \\ \mu_i^* g_i(x^*) &= 0 \quad \forall i && \text{(complementarità)} \end{aligned}$$

Intuizione 4.1. Al minimo vincolato, il gradiente di f deve trovarsi nel *cono normale* all'insieme ammissibile in x^* : non si può migliorare f lungo nessuna direzione ammissibile.

La condizione di complementarità $\mu_i^* g_i(x^*) = 0$ dice: o il vincolo g_i è inattivo (e il suo moltiplicatore è zero), oppure il moltiplicatore è positivo (e il vincolo è attivo, in equilibrio).

Teorema 4.3 (Condizioni del secondo ordine). *Siano soddisfatte le KKT in x^* . Definiamo il Lagrangiano ridotto:*

$$L(x) = f(x) + \sum_i \mu_i^* g_i(x) + \sum_j \lambda_j^* h_j(x).$$

- **Necessaria:** $d^\top \nabla^2 L(x^*) d \geq 0$ per ogni d nel cono critico.
- **Sufficiente:** $d^\top \nabla^2 L(x^*) d > 0$ per ogni $d \neq 0$ nel cono critico $\Rightarrow x^*$ minimo locale isolato.

4.2.2 Lemma di Farkas e dualità

Lemma 4.1 (Farkas). *Per $A \in \mathbb{R}^{m \times n}$, $c \in \mathbb{R}^n$, esattamente una delle seguenti è vera:*

1. *Esiste $x \geq 0$ con $Ax = c$.*
2. *Esiste y con $A^\top y \geq 0$ e $c^\top y < 0$.*

Il Lemma di Farkas è lo strumento fondamentale nella dimostrazione delle condizioni KKT.

4.3 Dualità Lagrangiana

Definizione 4.4 (Funzione Lagrangiana).

$$L(x, \mu, \lambda) = f(x) + \mu^\top g(x) + \lambda^\top h(x), \quad \mu \geq 0.$$

Definizione 4.5 (Funzione duale di Lagrange).

$$q(\mu, \lambda) = \inf_{x \in \mathbb{R}^n} L(x, \mu, \lambda).$$

q è sempre concava (anche se f, g, h non lo sono).

Teorema 4.4 (Weak duality). *Per ogni x ammissibile e $(\mu \geq 0, \lambda)$: $q(\mu, \lambda) \leq f^*$.*

Il problema duale $\max_{\mu \geq 0, \lambda} q(\mu, \lambda)$ fornisce sempre un lower bound su f^ .*

Definizione 4.6 (Duality gap). $f^* - q^* \geq 0$. Si ha *strong duality* quando $f^* = q^*$.

Teorema 4.5 (Condizione di Slater (strong duality per problemi convessi)). *Se il problema è convesso (f, g_i convesse, h_j affini) ed esiste $\hat{x}$ strettamente ammissibile ($g_i(\hat{x}) < 0$ per ogni i , $h_j(\hat{x}) = 0$ per ogni j), allora vale la strong duality: $f^* = q^*$.*

Esempio 4.1 (Dualità LP). $\min_x c^\top x$ s.t. $Ax = b, x \geq 0$ ha duale $\max_y b^\top y$ s.t. $A^\top y \leq c$. Per LP, la strong duality vale sempre (quando entrambi sono ammissibili).

Esempio 4.2 (Dualità QP). Per $\min \frac{1}{2} x^\top Q x + c^\top x$ s.t. $Ax = b, Q \succ 0$:

$$q(\lambda) = -\frac{1}{2} (c + A^\top \lambda)^\top Q^{-1} (c + A^\top \lambda) - b^\top \lambda.$$

Funzione duale quadratica concava, strong duality vale.

4.4 Algoritmi per l'ottimizzazione vincolata

4.4.1 Gradiente proiettato

Definizione 4.7 (Proiezione su insieme convesso). $\Pi_C(x) = \arg \min_{y \in C} \|y - x\|^2$. Unica per C convesso chiuso.

Algoritmo: Projected Gradient Descent

$$x_{k+1} = \Pi_S(x_k - \alpha_k \nabla f(x_k)).$$

Converge come GD standard se f è smooth e convessa, S è convesso.

Esempio 4.3 (Proiezioni facili). • Simpleso $\Delta = \{x \geq 0 : \sum x_i = 1\}$: $O(n \log n)$.

- Box $[l, u]$: $\Pi_{[l, u]}(x)_i = \max(l_i, \min(u_i, x_i))$.
- Palla $\|x\| \leq r$: $\Pi(v) = r \cdot v / \max(r, \|v\|)$.

4.4.2 Metodo di Frank-Wolfe (Conditional Gradient)

Algoritmo: Frank-Wolfe

1. Linearizza f in x_k : $s_k = \arg \min_{x \in C} \nabla f(x_k)^\top x$.
2. Aggiorna: $x_{k+1} = x_k + \alpha_k(s_k - x_k)$, $\alpha_k = 2/(k+2)$.

Costo per passo: solo la minimizzazione lineare su C (molto più facile della proiezione per certi insiemi, es. poliedri).

Teorema 4.6 (Convergenza Frank-Wolfe). Per f convessa L -smooth su C compatto (diametro D):

$$f(x_k) - f^* \leq \frac{2LD^2}{k+2}.$$

4.4.3 Metodi barriera (Interior Point)

Idea: approssima il vincolo $g(x) \leq 0$ con la *funzione barriera logaritmica* $-t \log(-g(x))$:

$$\min_x f(x) - t \sum_i \log(-g_i(x)) \quad (\text{nessun vincolo!}).$$

Si risolve una sequenza di problemi al diminuire di $t \rightarrow 0$. Il percorso delle soluzioni al variare di t è detto *central path*. I metodi interior point hanno complessità polinomiale e sono la scelta per LP/QP/SOCP di grande scala.

Parte II

Analisi Numerica Lineare (NLA)

Capitolo 5

Background: Algebra Lineare

5.1 Prodotto matriciale e interpretazioni

Osservazione 5.1 (Costo computazionale). Prodotto Av con $A \in \mathbb{R}^{m \times n}$: $O(mn)$ flop. Prodotto AB con $B \in \mathbb{R}^{n \times p}$: $O(mnp)$. **Regola fondamentale:** $A(Bv)$ costa $O(mn + np)$, mentre $(AB)v$ costa $O(mnp + mp)$ — non formare mai esplicitamente AB se si vuole solo ABv .

5.2 Sistemi triangolari

Un sistema $Lx = b$ con L triangolare inferiore si risolve con *forward substitution* in $O(n^2)$; $Ux = b$ con U triangolare superiore con *back substitution*.

Attenzione

Non calcolare mai A^{-1} esplicitamente per risolvere $Ax = b$: il costo è $O(n^3)$ (come LU) ma il risultato è meno accurato. Usare $A \backslash b$ in Matlab o `np.linalg.solve` in Python.

5.3 Autovalori e forme quadratiche

Teorema 5.1 (Teorema spettrale). Ogni $A = A^T \in \mathbb{R}^{n \times n}$ si decompone come $A = Q\Lambda Q^T$ con Q ortogonale, $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$, $\lambda_i \in \mathbb{R}$.

Definizione 5.1 (Definitezza). $Q \succ 0$ (definita positiva) $\Leftrightarrow \lambda_i > 0$ per ogni $i \Leftrightarrow x^T Q x > 0$ per ogni $x \neq 0$.

Proposizione 5.1. Per ogni $A \in \mathbb{R}^{m \times n}$: $A^T A \succeq 0$. Se A ha rango massimo ($\text{rank } A = n \leq m$): $A^T A \succ 0$.

Capitolo 6

Norme e Matrici Ortogonali

6.1 Norme vettoriali

Definizione 6.1. La famiglia delle p -norme: $\|x\|_p = (\sum_i |x_i|^p)^{1/p}$ per $p \geq 1$. La norma euclidea è $\|\cdot\|_2$; la norma $\|\cdot\|_\infty = \max_i |x_i|$; la norma $\|\cdot\|_1 = \sum_i |x_i|$.

6.2 Matrici ortogonali

Definizione 6.2. $U \in \mathbb{R}^{m \times m}$ è ortogonale se $U^\top U = UU^\top = I$.

Proposizione 6.1. Le matrici ortogonali preservano la norma euclidea: $\|Ux\|_2 = \|x\|_2$.
Conseguenza: i valori singolari di UA e AV (con U, V ortogonali) coincidono con quelli di A .

6.3 Norme di matrici

Definizione 6.3 (Norma indotta). La norma di matrice indotta dalla norma vettoriale $\|\cdot\|$ è:

$$\|A\| = \max_{\|x\|=1} \|Ax\| = \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

Teorema 6.1 (Norma spettrale). La norma indotta dalla norma euclidea è la norma spettrale:

$$\|A\|_2 = \sigma_1(A) = \max_i \sigma_i(A).$$

Per matrici ortogonali: $\|Q_1 A Q_2\|_2 = \|A\|_2$ (invarianza unitaria).

Definizione 6.4 (Norma di Frobenius). $\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2} = \sqrt{\sum_i \sigma_i^2}$. Anche la norma di Frobenius è unitariamente invariante.

Capitolo 7

Decomposizione a Valori Singolari (SVD)

7.1 Definizione e proprietà

Teorema 7.1 (SVD — Singular Value Decomposition). *Per ogni $A \in \mathbb{R}^{m \times n}$ (con $m \geq n$), esistono:*

- $U \in \mathbb{R}^{m \times m}$ ortogonale (vettori singolari sinistri).
- $V \in \mathbb{R}^{n \times n}$ ortogonale (vettori singolari destri).
- $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_n) \in \mathbb{R}^{m \times n}$ con $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$.

tali che $A = U\Sigma V^\top$.

Prova per A quadrata. La matrice $A^\top A$ è simmetrica semidefinita positiva. Sia $A^\top A = VDV^\top$ la sua decomposizione spettrale con $D = \text{diag}(\sigma_1^2, \dots, \sigma_n^2)$, $\sigma_i \geq 0$. Poniamo $\Sigma = \sqrt{D}$ e $U_r = AV_r\Sigma_r^{-1}$ (vettori singolari sinistri per i valori singolari non nulli). Si verifica che $U_r^\top U_r = \Sigma_r^{-1}V_r^\top A^\top AV_r\Sigma_r^{-1} = I$, e completando U_r a una base ortonormale si ottiene U . $\square$

Definizione 7.1 (SVD thin/ridotta). Per $A \in \mathbb{R}^{m \times n}$ con $m \geq n$, la *thin SVD* è $A = \hat{U}\Sigma V^\top$ con $\hat{U} \in \mathbb{R}^{m \times n}$ (colonne ortonormali). Sufficiente per la maggior parte delle applicazioni.

Teorema 7.2 (Sottospazi fondamentali). *Dalla SVD $A = U\Sigma V^\top$ con $\text{rank } A = r$:*

- $\text{Im}(A) = \text{span}(u_1, \dots, u_r)$ (prime r colonne di U).
- $\ker(A) = \text{span}(v_{r+1}, \dots, v_n)$ (ultimi $n - r$ colonne di V).
- $\text{Im}(A^\top) = \text{span}(v_1, \dots, v_r)$.
- $\ker(A^\top) = \text{span}(u_{r+1}, \dots, u_m)$.

Definizione 7.2 (Pseudoinversa di Moore-Penrose). $A^+ = V\Sigma^+U^\top$, dove $\Sigma^+ = \text{diag}(\sigma_1^{-1}, \dots, \sigma_r^{-1}, 0, \dots, 0)$.

7.2 Teorema di Eckart-Young (approssimazione di basso rango)

Teorema 7.3 (Eckart-Young, 1936). Sia $A = U\Sigma V^\top$ e sia $A_k = \sum_{i=1}^k \sigma_i u_i v_i^\top$ il troncamento al rango k . Allora:

$$\|A - A_k\|_2 = \sigma_{k+1}, \quad \|A - A_k\|_F = \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_n^2},$$

e A_k è il miglior approssimante di rango $\leq k$ di A in entrambe le norme.

Intuizione 7.1. A_k cattura le k “direzioni principali” di A (quelle con maggiore varianza/energia). Fondamentale per: PCA, compressione di immagini, sistemi di raccomandazione.

Esempio 7.1 (Matrice dei voti studenti). $A \in \mathbb{R}^{s \times t}$ con A_{ij} = voto dello studente i al test j . Se $A \approx A_1 = \sigma_1 u_1 v_1^\top$ (rango 1), allora $A_{ij} \approx \sigma_1 (u_1)_i (v_1)_j$: il voto è prodotto di una “abilità” dello studente per una “difficoltà” del test. Se $A \approx A_2$, ci sono due “dimensioni” di abilità.

Capitolo 8

Il Problema dei Minimi Quadrati

8.1 Formulazione

Definizione 8.1 (Least Squares). Dato $A \in \mathbb{R}^{m \times n}$ ($m \geq n$), $b \in \mathbb{R}^m$:

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2.$$

Se $m > n$, il sistema $Ax = b$ è sovradeterminato (più equazioni che incognite) — in generale non ha soluzione esatta.

Teorema 8.1 (Soluzione geometrica). x^* è soluzione LS $\Leftrightarrow$ il residuo $r^* = Ax^* - b$ è ortogonale a $\text{Im}(A)$, cioè $A^\top(Ax^* - b) = 0$.

Intuizione 8.1. Ax^* è la proiezione ortogonale di b su $\text{Im}(A)$. Il punto più vicino a b nell'immagine di A si trova proiettando.

8.2 Equazioni normali

Teorema 8.2 (Equazioni normali). x^* è soluzione LS $\Leftrightarrow$ soddisfa le equazioni normali:

$$A^\top Ax^* = A^\top b.$$

Se A ha rango massimo ($\text{rank } A = n$, cioè $A^\top A \succ 0$), la soluzione è unica: $x^* = (A^\top A)^{-1} A^\top b$.

8.3 Soluzione via SVD

Dalla thin SVD $A = \hat{U}\Sigma V^\top$, la soluzione di norma minima è:

$$x^* = A^+ b = V \Sigma^+ \hat{U}^\top b = \sum_{i=1}^r \frac{\hat{u}_i^\top b}{\sigma_i} v_i.$$

Osservazione 8.1 (Valori singolari piccoli e regolarizzazione). Se A ha valori singolari piccoli, la soluzione x^* è amplificata enormemente. La *regolarizzazione di Tikhonov* (ridge regression)

modifica il problema:

$$\min_x \|Ax - b\|^2 + \delta \|x\|^2 \quad \Rightarrow \quad x_\delta = (A^\top A + \delta I)^{-1} A^\top b = V \operatorname{diag}\left(\frac{\sigma_i}{\sigma_i^2 + \delta}\right) \hat{U}^\top b.$$

Per $\delta > 0$, i valori singolari piccoli vengono smorzati: $\sigma_i/(\sigma_i^2 + \delta) \approx 1/\delta$ se $\sigma_i \ll \sqrt{\delta}$.

Capitolo 9

Fattorizzazione QR

9.1 Definizione e teorema

Teorema 9.1 (Fattorizzazione QR). *Per ogni $A \in \mathbb{R}^{m \times n}$ con $m \geq n$, esistono:*

- $Q \in \mathbb{R}^{m \times m}$ ortogonale.
- $R \in \mathbb{R}^{m \times n}$ triangolare superiore.

tali che $A = QR$. La thin QR è $A = \hat{Q}R$ con $\hat{Q} \in \mathbb{R}^{m \times n}$ (colonne ortonormali) e $R \in \mathbb{R}^{n \times n}$ triangolare superiore.

9.2 Riflettori di Householder

Il metodo numericamente stabile per calcolare la QR.

Definizione 9.1 (Riflettore di Householder). Dato un vettore $v \in \mathbb{R}^m$, il riflettore di Householder è:

$$H = I - 2 \frac{vv^\top}{\|v\|^2} \in \mathbb{R}^{m \times m}.$$

H è simmetrica e ortogonale ($H = H^\top$, $H^2 = I$). Riflette rispetto all'iperpiano ortogonale a v .

Proposizione 9.1 (Azzeramento di colonne). *Dato $x \in \mathbb{R}^m$, si sceglie $v = x - \|x\|e_1$ (o $v = x + \|x\|e_1$ per stabilità numerica, scegliendo il segno opposto a x_1). Allora:*

$$Hx = \|x\| e_1 = (\|x\|, 0, \dots, 0)^\top.$$

Questo azzerava tutti gli elementi sotto il primo.

Algoritmo: Fattorizzazione QR via Householder

Per $j = 1, \dots, n$:

1. Considera $x = A_{j:m,j}$ (colonna corrente a partire dalla diagonale).
2. Costruisci v_j tale che $H_j x = \|x\| e_1$.

3. Aggiorna: $A \leftarrow H_j A$ (applica il riflettore all'intera matrice).

Alla fine: $H_n \cdots H_1 A = R$ (triangolare sup.), quindi $Q = H_1 \cdots H_n$. **Costo:** $2mn^2 - \frac{2}{3}n^3$ flop.

Teorema 9.2 (Backward stability della QR). *La fattorizzazione QR calcolata con riflettori di Householder soddisfa:*

$$\tilde{Q}\tilde{R} = A + \Delta A, \quad \frac{\|\Delta A\|_F}{\|A\|_F} = O(u),$$

dove $u \approx 2.2 \times 10^{-16}$ è la precisione di macchina. La QR di Householder è backward stable.

9.3 Soluzione dei minimi quadrati con QR

Con la thin QR $A = \hat{Q}R$:

$$\|Ax - b\|^2 = \|\hat{Q}Rx - b\|^2 = \|Rx - \hat{Q}^\top b\|^2 + \|(I - \hat{Q}\hat{Q}^\top)b\|^2.$$

Il secondo termine non dipende da x . Quindi si risolve il sistema triangolare:

$$Rx^* = \hat{Q}^\top b.$$

Costo totale: QR in $O(mn^2)$, risoluzione $Rx = c$ in $O(n^2)$.

Attenzione

Le equazioni normali $A^\top Ax = A^\top b$ risolte con Cholesky hanno costo simile, ma l'errore numerico è proporzionale a $\kappa(A)^2$ anziché $\kappa(A)$. Preferire QR per problemi mal condizionati.

Capitolo 10

Condizionamento e Stabilità

10.1 Numero di condizionamento

Definizione 10.1 (Numero di condizionamento assoluto e relativo). Per $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$:

$$\kappa_{\text{abs}}(f, x) = \lim_{\delta \rightarrow 0} \sup_{\|d\| \leq \delta} \frac{\|f(x+d) - f(x)\|}{\|d\|}, \quad \kappa_{\text{rel}}(f, x) = \kappa_{\text{abs}} \cdot \frac{\|x\|}{\|f(x)\|}.$$

Teorema 10.1 (Numero di condizionamento di un sistema lineare). Per $f(b) = A^{-1}b$ (risolvere $Ax = b$):

$$\kappa(A) = \|A\| \cdot \|A^{-1}\| = \frac{\sigma_1}{\sigma_n}.$$

Perturbando b di δb : $\frac{\|\delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\delta b\|}{\|b\|}$.

Intuizione 10.1. $\kappa(A) \gg 1$: problema *mal condizionato*. Se $\kappa(A) = 10^6$ e b ha un errore relativo di 10^{-12} (precisione di macchina), la soluzione x ha un errore relativo di $\approx 10^{-6}$: si perdono 6 cifre decimali.

Proposizione 10.1 (Distanza dalla singolarità). $1/\kappa(A) = \min_{\tilde{A} \text{ singolare}} \|\tilde{A} - A\|/\|A\|$. Matrici con $\kappa(A) \gg 1$ sono “quasi singolari”.

Teorema 10.2 (Condizionamento del problema LS). Per $\min_x \|Ax - b\|$ con A a rango massimo e soluzione x^* :

- Perturbazione di b : errore relativo proporzionale a $\kappa(A)$.
- Perturbazione di A : errore proporzionale a $\kappa(A)^2$ se $\|r^*\|/\|b\|$ non è trascurabile.

Nota: $\kappa(A^\top A) = \kappa(A)^2$, confermando l'instabilità delle equazioni normali.

10.2 Aritmetica floating point

I numeri double-precision (IEEE 754, 64 bit) hanno struttura:

$$\underbrace{1 \text{ bit}}_{\text{segno}} \underbrace{11 \text{ bit}}_{\text{esponente}} \underbrace{52 \text{ bit}}_{\text{mantissa}}.$$

- Range: $\approx [10^{-308}, 10^{308}]$.
- Precisione di macchina: $u = 2^{-52} \approx 2.2 \times 10^{-16}$.
- Ogni numero reale x è rappresentato da $\tilde{x}$ con $|\tilde{x} - x|/|x| \leq u$.
- Ogni operazione aritmetica introduce un errore relativo $\leq u$: $a \oplus b = (a + b)(1 + \delta)$, $|\delta| \leq u$.

Osservazione 10.1 (Problemi di cancellazione). Se si sottraggono due numeri quasi uguali, le cifre significative vengono annullate. Es. $1.000001 - 1.000000 = 0.000001$: con 7 cifre significative, il risultato ne ha solo 1. Algoritmi che evitano la cancellazione sono preferibili.

10.3 Stabilità degli algoritmi

Definizione 10.2 (Backward stability). Un algoritmo che calcola $\tilde{y} \approx f(x)$ è *backward stable* se:

$$\tilde{y} = f(\tilde{x}), \quad \frac{\|\tilde{x} - x\|}{\|x\|} = O(u).$$

L'output è l'esatto risultato per un input *leggermente perturbato*.

Teorema 10.3 (Backward stability del prodotto interno). Sia $y = a^\top b = \sum_{i=1}^n a_i b_i$ calcolato in macchina come $\tilde{y}$. Allora:

$$\tilde{y} = \tilde{a}^\top b = a^\top \tilde{b},$$

dove $\|\tilde{a} - a\| \leq nu\|a\| + O(u^2)$ e analogamente per b . Il prodotto interno è backward stable.

Prova (sketch). Ogni moltiplicazione $a_i \odot b_i = a_i b_i (1 + \delta_i)$, $|\delta_i| \leq u$. Le addizioni successive introducono ulteriori errori. Un'analisi rigorosa mostra che l'errore accumulato soddisfa:

$$|\tilde{y} - y| \leq nu \sum_i |a_i b_i| + O(u^2) \leq nu\|a\|_1 \|b\|_\infty.$$

L'algoritmo può essere visto come il calcolo esatto di $\tilde{a}^\top b$ con $\tilde{a}_i = a_i(1 + \varepsilon_i)$, $|\varepsilon_i| \leq (2i - 1)u$: backward stable. $\square$

Esempio 10.1 (Prodotto interno di vettori lunghi). Con $n = 1000$ e $u \approx 10^{-16}$: l'errore relativo del prodotto interno è $\leq 1000 \cdot 10^{-16} = 10^{-13}$. Ancora molto piccolo! Il prodotto interno è numericamente robusto.

Teorema 10.4 (Backward stability di Householder QR). La fattorizzazione QR tramite riflettori di Householder è backward stable: $\tilde{Q}\tilde{R} = A + E$ con $\|E\| = O(u\|A\|)$.

Attenzione

Il metodo di Gram-Schmidt classico **non** è backward stable: gli errori di cancellazione possono rendere le colonne di Q non ortogonali. Gram-Schmidt modificato è meglio, ma Householder rimane il metodo preferito.

10.3.1 Test a posteriori: analisi del residuo

Teorema 10.5 (Residual bound per sistemi lineari). *Dato $\tilde{x} \approx A^{-1}b$ con residuo $r = A\tilde{x} - b$:*

$$\frac{\|\tilde{x} - x^*\|}{\|x^*\|} \leq \kappa(A) \cdot \frac{\|r\|}{\|b\|}.$$

Se $\|r\|/\|b\| = O(u)$ (residuo piccolo in machine precision), allora l'algoritmo è backward stable. Ma l'errore forward può ancora essere grande se $\kappa(A) \gg 1$.

Capitolo 11

Metodi Iterativi per Sistemi Lineari

11.1 Motivazione: sistemi sparsi di grande scala

Per $n = 10^5$ – 10^7 : LU richiede $O(n^3) \gg 10^{15}$ flop — completamente impraticabile. Ma spesso la matrice è *sparsa*: $\text{nnz}(A) \ll n^2$ (es. $O(n)$ nonzeri per riga). Il prodotto Av ha allora costo $O(\text{nnz}(A))$.

I *metodi iterativi di Krylov* sfruttano questa struttura: ogni passo richiede solo un prodotto matrice-vettore.

11.2 Spazi di Krylov

Definizione 11.1. $\mathcal{K}_k(A, b) = \text{span}(b, Ab, A^2b, \dots, A^{k-1}b) \subseteq \mathbb{R}^n$.

Proposizione 11.1. $x_k = q_{k-1}(A)b$ per qualche polinomio q_{k-1} di grado $\leq k-1$. I metodi di Krylov scelgono q_{k-1} in modo ottimale rispetto a qualche criterio.

11.3 Gradiente Coniugato per sistemi SPD

CG (già visto nell'ottimizzazione) risolve $Qx = v$ con $Q \succ 0$:

Teorema 11.1 (Convergenza CG).

$$\frac{\|x_k - x^*\|_Q}{\|x_0 - x^*\|_Q} \leq 2 \left(\frac{\sqrt{\kappa(Q)} - 1}{\sqrt{\kappa(Q)} + 1} \right)^k.$$

Se Q ha m autovalori distinti, CG converge in esattamente m passi. Per autovalori raggruppati (*clustered*), la convergenza è rapida: il polinomio di Chebyshev approssima bene la “forma” dello spettro.

11.3.1 Precondizionamento

Definizione 11.2 (Preconditioned CG). $P \approx Q$ invertibile, $P \succ 0$. Si risolve $P^{-1}Qx = P^{-1}v$, che ha lo stesso spettro “rimodellato”.

La convergenza dipende da $\kappa(P^{-1}Q) \ll \kappa(Q)$. Scelte comuni:

- $P = \text{diag}(Q)$ (Jacobi): semplice, smorzamento delle scale.
- P = fattore di Cholesky incompleto (ICC): più efficace.
- $P = \text{ILUTP}$ per sistemi non simmetrici.

11.4 Algoritmo di Arnoldi

Per sistemi non simmetrici, si costruisce una base ortonormale di $\mathcal{K}_k(A, b)$ stabilmente.

Algoritmo: Arnoldi

$q_1 = b/\|b\|$. Per $j = 1, \dots, k$:

1. $w = Aq_j$.
2. Per $i = 1, \dots, j$: $h_{ij} = q_i^\top w$; $w \leftarrow w - h_{ij}q_i$ (ortogonalizzazione).
3. $h_{j+1,j} = \|w\|$; se $h_{j+1,j} = 0$: STOP (breakdown). Altrimenti $q_{j+1} = w/h_{j+1,j}$.

Teorema 11.2 (Fattorizzazione di Arnoldi). *Dopo k passi: $AQ_k = Q_{k+1}H_k$, dove:*

- $Q_k = [q_1, \dots, q_k] \in \mathbb{R}^{n \times k}$ ha colonne ortonormali.
- $H_k \in \mathbb{R}^{(k+1) \times k}$ è una matrice di Hessenberg superiore.

Intuizione 11.1. Arnoldi è Gram-Schmidt applicato alle potenze di A : costruisce una base ortogonale senza formare esplicitamente $[b, Ab, \dots, A^{k-1}b]$ (che sarebbero numericamente instabili per potenze elevate).

Osservazione 11.1 (Caso simmetrico: Lanczos). Se $A = A^\top$, la matrice H_k è simmetrica tridiagonale. L'algoritmo si semplifica nella ricorrenza di Lanczos: ogni passo richiede solo 2 prodotti matrice-vettore e la memorizzazione di 3 vettori (invece di k per Arnoldi). Si ricade nel metodo CG per sistemi SPD.

11.5 GMRES

Definizione 11.3 (GMRES — Generalized Minimal Residual). Al passo k , GMRES trova la migliore approssimazione nello spazio di Krylov rispetto al residuo:

$$x_k = \arg \min_{x \in x_0 + \mathcal{K}_k(A, r_0)} \|Ax - b\|.$$

Algoritmo: GMRES

1. Esegui k passi di Arnoldi su (A, r_0) : ottieni Q_{k+1}, H_k .
2. Nota che $r_k = Ax_k - b = Q_{k+1}(H_k z_k - \|r_0\|e_1)$ con $x_k = x_0 + Q_k z_k$.
3. Risolvi il piccolo problema LS: $z_k = \arg \min_z \|H_k z - \|r_0\|e_1\|$ (costo $O(k^2)$).

4. Soluzione: $x_k = x_0 + Q_k z_k$.

Il residuo si calcola come by-product dalla fattorizzazione QR di H_k (aggiornata a ogni passo).

Teorema 11.3 (Convergenza GMRES). *Se $A = V\Lambda V^{-1}$ (diagonalizzabile), allora:*

$$\frac{\|r_k\|}{\|r_0\|} \leq \kappa(V) \min_{p \in \mathcal{P}_k, p(0)=1} \max_{\lambda_i} |p(\lambda_i)|,$$

dove $\mathcal{P}_k$ è l'insieme dei polinomi di grado $\leq k$. Il termine $\kappa(V)$ misura quanto A “non è normale”.

Intuizione 11.2. GMRES converge rapidamente se gli autovalori di A sono *raggruppati* in pochi cluster: il polinomio ottimale può essere piccolo su tutti gli autovalori con poche “radici” (una per cluster). Se A ha autovalori ben separati, la convergenza è lenta.

Breakdown fortunato: se $\|w\| = 0$ in Arnoldi, allora $Ax = b$ ha una soluzione esatta nel Krylov — GMRES ha trovato la soluzione!

Attenzione

GMRES ha costo crescente per iterazione: $O(k^2)$ per la fattorizzazione QR di H_k e $O(kn)$ per lo storage. Si usa *restarted GMRES*: dopo m passi si ricomincia da x_m . Il costo diventa $O(mn)$ per ciclo, ma la convergenza può rallentare.

Capitolo 12

Fattorizzazioni Dirette per Sistemi Lineari

12.1 Fattorizzazione LU

Teorema 12.1 (LU con pivoting parziale). *Per ogni $A \in \mathbb{R}^{n \times n}$, con un'opportuna matrice di permutazione P :*

$$PA = LU,$$

*con L triangolare inferiore con diagonale unitaria ($|L_{ij}| \leq 1$), U triangolare superiore. **Costo:** $\frac{2}{3}n^3$ flop.*

Intuizione 12.1. LU è la fattorizzazione dell'eliminazione gaussiana. Il pivoting parziale (scegliere il pivot come l'elemento di modulo massimo nella colonna corrente) garantisce la stabilità numerica: $|L_{ij}| \leq 1$ previene la crescita degli errori di arrotondamento.

Algoritmo: Risoluzione via LU

Per $Ax = b$ con $PA = LU$:

1. Permuta: $\tilde{b} = Pb$.
2. Forward substitution: $Ly = \tilde{b}$, $O(n^2)$.
3. Back substitution: $Ux = y$, $O(n^2)$.

Costo totale: $\frac{2}{3}n^3$ (fattorizzazione) + $2n^2$ (soluzione). Stessa fattorizzazione per più termini noti.

Teorema 12.2 (Backward stability di LU con pivoting). *LU con pivoting parziale è backward stable: $\tilde{L}\tilde{U} = PA + E$ con $\|E\| = O(u\|A\|)$ (tipicamente — casi patologici esistono ma sono rari in pratica).*

12.2 Fattorizzazione $LDL^\top$ per matrici simmetriche

Per $M = M^\top$, l'eliminazione simmetrica produce $M = LDL^\top$: costo $\frac{1}{3}n^3$ (metà di LU pieno sfruttando la simmetria). Il pivoting di Bunch-Kaufman usa blocchi 1×1 e 2×2 in D per stabilità.

12.3 Fattorizzazione di Cholesky

Teorema 12.3 (Cholesky). *Se $M = M^\top \succ 0$, esiste un'unica R triangolare superiore con diagonale positiva tale che $M = R^\top R$. Costo: $\frac{1}{3}n^3$ — metà di LU!*

Proposizione 12.1. $M \succ 0 \Leftrightarrow$ tutti i pivot dell'eliminazione di Cholesky sono positivi. Quindi Cholesky può servire come test di definitezza positiva (se fallisce, $M \not\succ 0$).

Algoritmo: Risoluzione con Cholesky

Per $Mx = b$ con $M \succ 0$:

1. Calcola $R = \text{chol}(M)$, $\frac{1}{3}n^3$.
2. Forward: $R^\top y = b$, $O(n^2)$.
3. Back: $Rx = y$, $O(n^2)$.

12.4 Confronto e scelta dell'algoritmo

Struttura di A	Algoritmo	Costo	Backward stable
Quadrata generica	LU + pivoting parz.	$\frac{2}{3}n^3$	Sì (praticamente)
Simmetrica	$LDL^\top$ (Bunch-K.)	$\frac{1}{3}n^3$	Sì
SPD	Cholesky	$\frac{1}{3}n^3$	Sì (no pivoting!)
LS, $m \geq n$	QR (Householder)	$2mn^2 - \frac{2}{3}n^3$	Sì
SPD sparsa	CG (precondizionato)	$O(k \cdot \text{nnz})$	—
Generale sparsa	GMRES (precond.)	$O(k \cdot \text{nnz})$	—

Osservazione 12.1 (Regola pratica). Per sistemi densi: scegliere in base alla struttura (Cholesky se SPD, LU altrimenti). Per sistemi sparsi di grande scala ($n > 10^4$): usare CG (se SPD) o GMRES (altrimenti) con un buon preconditionatore. Non usare mai $\text{inv}(A)$.

12.5 Esempi pratici

Esempio 12.1 (Riordino per ridurre il fill-in). Per matrici sparse, LU/Cholesky può produrre fattori molto più densi (*fill-in*). Riordinare con Cuthill-McKee (`symrcm` in Matlab) riduce l'ampiezza di banda.

```
1 p = symrcm(A);
2 R = chol(A(p, p));           % Cholesky riordinato
3 x_perm = R \ (R' \ b(p));
4 x(p) = x_perm;              % permutazione inversa
```

Esempio 12.2 (Tikhonov via SVD in Python).

```
1 import numpy as np
2 U, s, Vt = np.linalg.svd(A, full_matrices=False)
3 delta = 1e-4
4 d = s / (s**2 + delta)      # filtro di Tikhonov
5 x = Vt.T @ (d * (U.T @ b)) # soluzione regolarizzata
```


Capitolo 13

Riepilogo: Connessioni tra NLA e Ottimizzazione

Le due parti del corso sono strettamente interconnesse:

1. **CG è sia ottimizzazione che algebra lineare:** risolvere $Qx = v$ con $Q \succ 0$ equivale a minimizzare $\frac{1}{2}x^\top Qx - v^\top x$. Stessa ricorrenza, due interpretazioni.
2. **Il passo di Newton richiede un sistema lineare:** $H_f(x_k)d_k = -\nabla f(x_k)$. Per sistemi grandi, Newton “inesatto” usa CG o GMRES per risolvere il sistema approssimativamente.
3. **Le equazioni normali del LS sono SPD:** $A^\top Ax = A^\top b$, risolto con Cholesky $O(\frac{1}{3}n^3)$, ma QR è preferibile per stabilità (κ^2 vs κ).
4. **SVD e PCA:** la decomposizione a valori singolari è lo strumento fondamentale per la compressione dei dati, il calcolo della pseudoinversa, e l’analisi della sensitività.
5. **Sistemi KKT:** i punti stazionari di problemi con vincoli di uguaglianza soddisfano il sistema saddle-point:

$$\begin{pmatrix} H & A^\top \\ A & 0 \end{pmatrix} \begin{pmatrix} x \\ \lambda \end{pmatrix} = \begin{pmatrix} -\nabla f \\ 0 \end{pmatrix}.$$

Questo è un sistema lineare simmetrico ma indefinito (non SPD): richiedono $LDL^\top$ con pivoting.

6. **Regolarizzazione di Tikhonov:** il problema di regressione $\|Ax - b\|^2 + \delta\|x\|^2$ è un problema LS regolarizzato. La soluzione si ottiene con SVD: $x_\delta = V \text{diag}(\sigma_i/(\sigma_i^2 + \delta))U^\top b$. Il parametro δ bilancia fit dei dati e norma della soluzione (bias-variance tradeoff in ML).

Appendice A

Appendice: Richiami e Tabelle di Riferimento

A.1 Notazione

Simbolo	Significato
$A \in \mathbb{R}^{m \times n}$	Matrice $m \times n$ reale
$A^\top$	Trasposta
A^{-1}	Inversa (A quadrata non singolare)
A^+	Pseudoinversa di Moore-Penrose
$\ v\ _2$	Norma euclidea
$\ A\ _2$	Norma spettrale $= \sigma_1(A)$
$\ A\ _F$	Norma di Frobenius $= \sqrt{\sum \sigma_i^2}$
$\kappa(A)$	Numero di condizionamento $= \sigma_1/\sigma_n$
$\sigma_i(A)$	i -esimo valore singolare
$\lambda_i(A)$	i -esimo autovalore
$Q \succ 0$	Q definita positiva
$Q \succeq 0$	Q semidefinita positiva
$\mathcal{K}_k(A, b)$	Spazio di Krylov di ordine k
$u \approx 2.2 \times 10^{-16}$	Precisione di macchina (double)
$\partial f(x)$	Subdifferenziale di f in x
$\text{prox}_{\alpha f}(v)$	Operatore prossimale

A.2 Costi computazionali principali

Operazione	Costo (flop)	Note
$Av, A \in \mathbb{R}^{m \times n}$	$2mn$	matrice-vettore
$A^\top A$	mn^2	non fare se evitabile
Sostituzione triangolare $n \times n$	n^2	forw./back. subst.
LU (con pivoting)	$\frac{2}{3}n^3$	
Cholesky	$\frac{1}{3}n^3$	A SPD
QR (thin, $m \geq n$)	$2mn^2 - \frac{2}{3}n^3$	Householder
SVD (thin, $m \geq n$)	$O(mn^2)$	
Eigendecomp. $n \times n$	$O(n^3)$	
CG, k passi	$k \cdot (2 \text{nnz}(A) + O(n))$	
GMRES, k passi	$k \cdot (2 \text{nnz}(A) + O(kn))$	storage cresce!

A.3 Tassi di convergenza degli algoritmi di ottimizzazione

Metodo	Tasso	Ipotesi
GD, passo fisso	$O(L/k)$	f convessa L -smooth
GD, passo fisso	$\left(1 - \frac{1}{\kappa}\right)^k$	f str. convessa
GD, quadratica	$\left(\frac{\kappa-1}{\kappa+1}\right)^k$	f quadratica, α ottimo
Nesterov/FISTA	$O(L/k^2)$	f convessa L -smooth
CG	$2 \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k$	$Q \succ 0$
Newton (locale)	$O(\ e_k\ ^2)$ quadratica	$f \in C^3$, vicino a x^*
BFGS	$o(\ e_k\)$ superlineare	f smooth str. conv.
Subgradiente	$O(G/\sqrt{k})$	f convessa G -Lipschitz
ISTA/prox grad.	$O(L/k)$	$f = h + r$, h L -smooth

A.4 Regole pratiche per l'esame

Scelta dell'algoritmo di ottimizzazione

- f smooth convessa, n piccolo: GD o Newton.
- f smooth convessa, n grande: L-BFGS o CG (per quadratiche).
- $f = h + r$ separabile smooth+nonsmooth: ISTA/FISTA (proximal gradient).

- f nonsmooth in generale: metodo del subgradiente o bundle.
- Problema vincolato convesso: gradiente proiettato, Frank-Wolfe, o interior point.

Scelta del metodo per sistemi lineari $Ax = b$

- A SPD, densa: Cholesky ($\frac{1}{3}n^3$).
- A simmetrica indef., densa: $LDL^\top$.
- A generica, densa: LU con pivoting.
- A SPD sparsa: PCG (CG preconditionato).
- A sparsa non simm.: GMRES preconditionato.
- LS $\|Ax - b\|$, A densa: QR (preferire a equazioni normali).
- LS con dati noisy: regolarizzazione Tikhonov via SVD.

Bibliografia

- [1] L.N. Trefethen e D. Bau III, *Numerical Linear Algebra*, SIAM, 1997.
- [2] J.W. Demmel, *Applied Numerical Linear Algebra*, SIAM, 1997.
- [3] A. Frangioni, *Optimization & Learning — Lecture Notes 2025/26*, Università di Pisa, 2025.
- [4] D.P. Bertsekas, *Nonlinear Programming*, 3rd ed., Athena Scientific, 2016.
- [5] J. Nocedal e S.J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006.
- [6] S. Boyd e L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.
- [7] G. Strang, *Linear Algebra and Learning from Data*, Wellesley-Cambridge Press, 2019.
- [8] S. Bubeck, *Convex Optimization: Algorithms and Complexity*, arXiv:1405.4980, 2015.